

Computer Science & IT

Database Management System



Transactions & concurrency control

Lecture No. 02

By- Vishal Sir

Recap of Previous Lecture



✓ Topic

Transaction

✓ Topic

ACID properties

✓ Topic

Atomicity

✓ Topic

Durability

Topics to be Covered



Topic

Isolation



Topic

Schedule



Topic

Equivalent schedule



Topic

Serializable schedule



Topic

Consistency





Topic : Transaction



A sequence of logically related operation which is used to perform a particular task is called a transaction.

Read(A) :-
Read of data item 'A'

This operation is used to read the value of dataitem 'A' from the database (ie. from the disk) into Main Mem.

Write(B) :-

Update the new value of dataitem B into the database, Writ. value of B present in Main memory

let initially , amount in account 'A' = 1000/-

and amount in account 'B' = 0/-

Write the transaction to transfer Rs 500/- from account A to account B.

Transaction

Read (A); : Because of this opⁿ a copy of data item 'A' will be created in M.M.

A = A - 500; // This updation will be performed only in M.M.

Write (A);

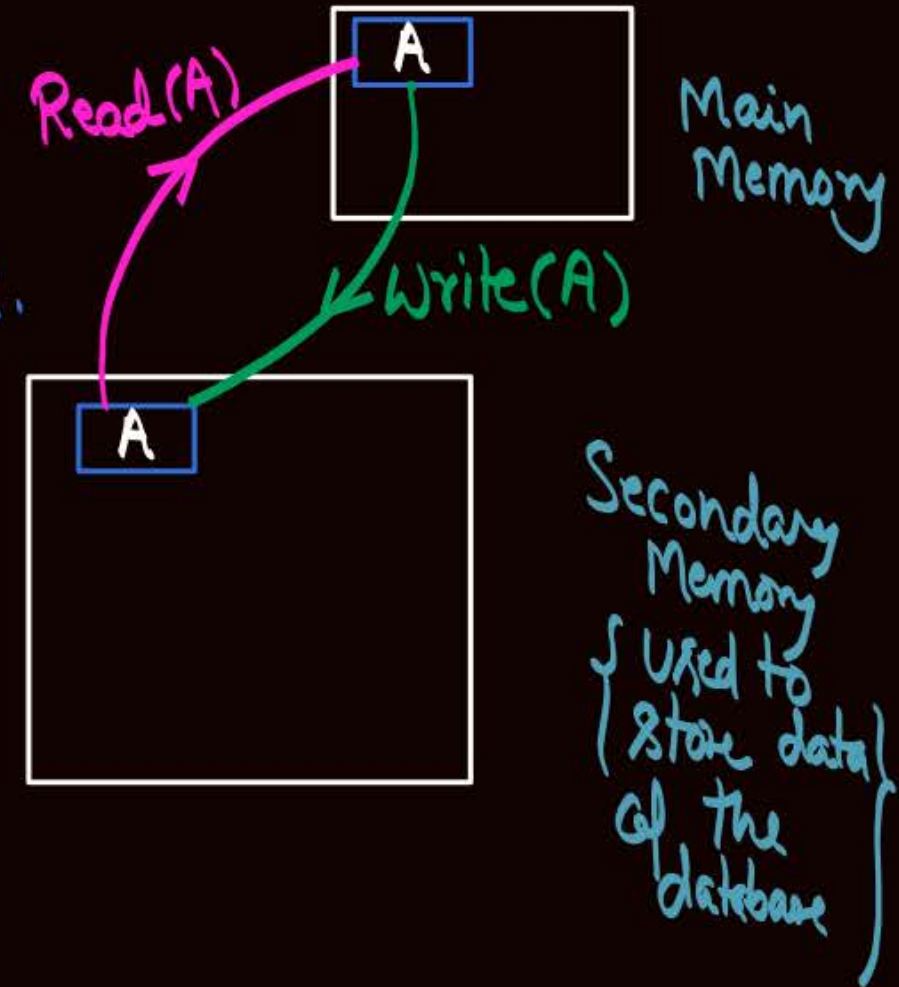
Read (B);

B = B + 500;

Write (B);

Commit; { Transaction Committed means transaction executed successfully from user point of view }

These two operations are not important from database point of view.





Topic : Blind write



Transaction(T)

R(A)

A = A - 10;

W(A)

W(B);

← It is not the blind write opⁿ, because T has already read the dataitem 'A' before it updates 'A'

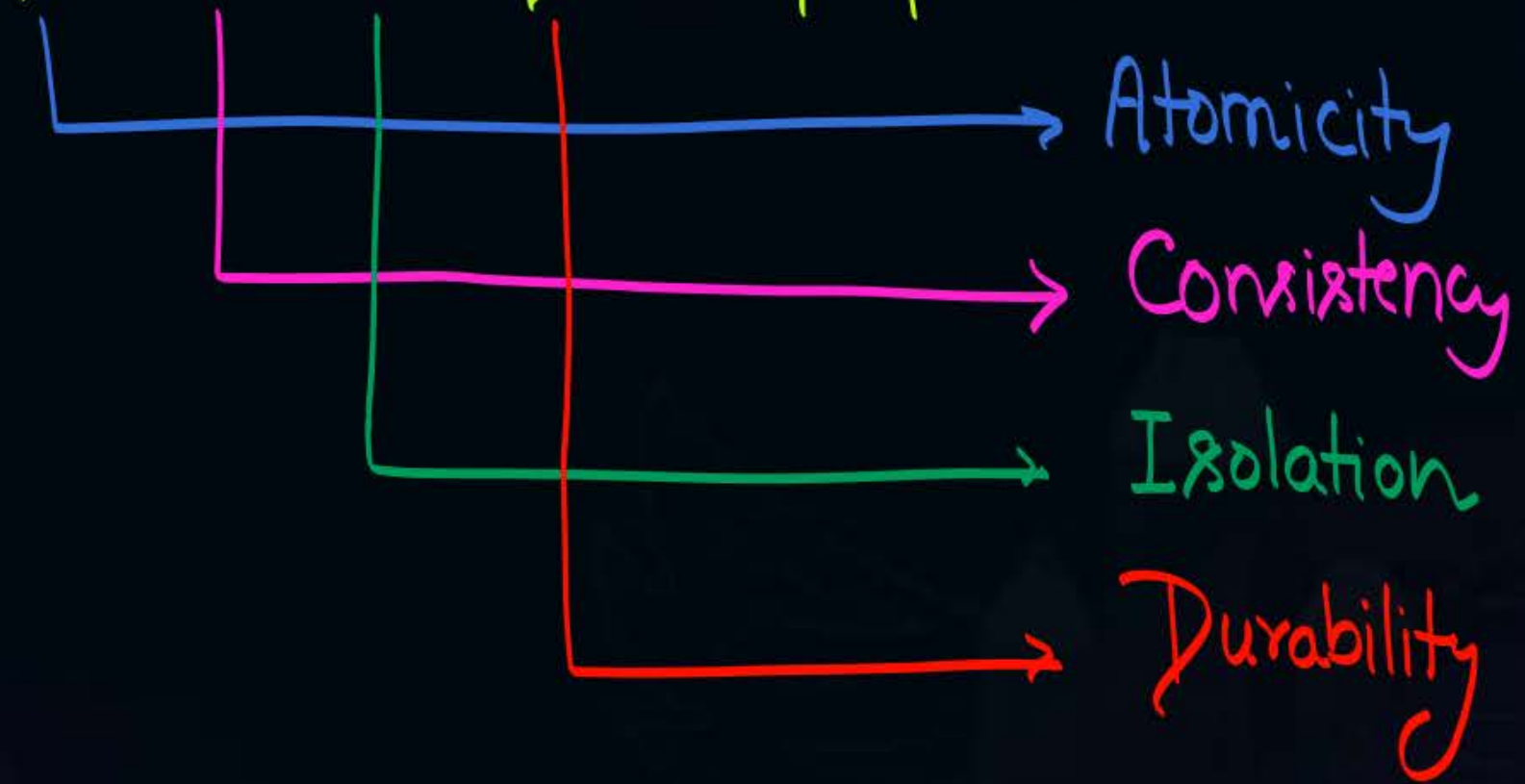
{ Transaction T did not read the data-item 'B', and it is trying to update the dataitem 'B' without reading the value of dataitem B from the database. Such write opⁿ are called "Blind write" opⁿ



Topic : ACID properties



For the integrity of the database the transactions of database must satisfy A C I D Properties.





Topic : Atomicity



Atomicity states that either execute all operations of a transaction or execute none of them.

There are two problems:-

① All the operations of a transaction can not be executed at once, they will get executed in a sequence one after another.

② If we start executing the operations of a transaction then we can not guarantee that the transaction will not fail in between { i.e., transaction may fail after executing some of the operations }

let initially , amount in account 'A' = 1000/-

and amount in account 'B' = 0/-

Write the transaction to transfer Rs 500/- from account A to account B.

Transaction

Read (A);

$A = A - 500;$

Write (A);

Read (B);

$B = B + 500;$

Write (B);

Commit; { Transaction Committed means transaction executed successfully from user point of view }

If we start executing the operations of this transaction and if transaction fails anywhere before successfully executing the Commit opn then atomicity is violated.

Reason of failure may be anything.

eg

- (i) Power failure
- (ii) Software Crash
- (iii) Disk Crash
- (iv) Any natural calamity
- (v) OS or DBMS may kill a transaction if required.

etc.,

⋮

Note :- →

To ensure atomicity, if a transaction fails before executing its commit operation successfully, then we must "undo" the changes performed by that transaction before its failure point.

{ If we can do so, the behaviour will be similar to the situation when none of the operation of that transaction is executed }

To ensure atomicity "Recovery Management Component" is used by DBMS.



Topic : Recovery management component

- * Recovery management Component is responsible for "undo" and "redo" operation

"Redo" opn will be performed w.r.t. Committed transaction.

"undo" opn will be performed w.r.t. failed transactions.

- * The process of "undoing" the operations performed by a failed transaction is called "rollback" of that transaction
- * For the purpose of "undo" and "redo" operation, "transaction log" is maintained by recovery management Component.



Topic : Transaction log

- * "Transaction log" is used to record the activities performed by the transactions.

let initially, amount in account 'A' = 1000/-
and amount in account 'B' = 0/-

Write the transaction to transfer Rs 500/- from account A to account B.

Transaction

Read (A);

$A = A - 500;$

Write (A);

Read (B);

$B = B + 500;$

Write (B);

Commit;

Fails → *

"Transaction log" W.r.t transaction to transfer Rs 500/- from account A to account B

Transaction log

A.old = 1000

A.new = 500

← transaction fails

In order to rollback the transaction, we must update the values in the database by the old values present in the transaction log for those data items.



Topic : Transaction log

* "Transaction log" is used to record the activities performed by the transactions.

→ We can recover from a failure only if transaction log is available,

o Transaction log is maintained by recovery management component into the secondary mem.

* Either we can maintain a separate transaction log (or) we can maintain a unified transaction log for each ongoing transaction.

Consider 3 transactions, T₁, T₂, T₃

(T, X, P, Q)

transaction id data item old value new value

T₁ Start;
(T₁, A, 50, 100)
(T₁, B, 70, 50)
T₂ Start;
(T₂, C, 100, 200)
Check point;
(T₁, D, 200, 100)
(T₂, E, 50, 500)
T₂ Commit;
T₃ Start;
(T₃, F, 100, 550)

We don't need to "redo" the opⁿ performed by T₂ before last "Check point".

The opⁿ before the Check point have been reflected into DB.

We will redo this opⁿ

Because T₂ Committed

Failure

T₁ & T₃ were Ongoing opⁿ When failure occurred
∴ We must "undo" all the operations Performed by T₁ & T₃.



Topic : Durability



- ★ Changes performed by committed transaction must persist in the database even in the case of failure.



Topic : Isolation

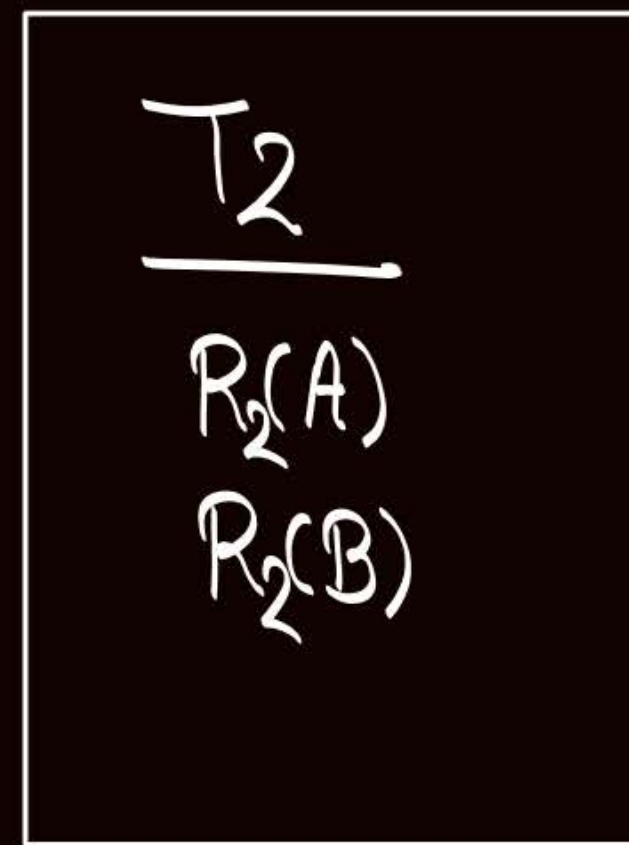
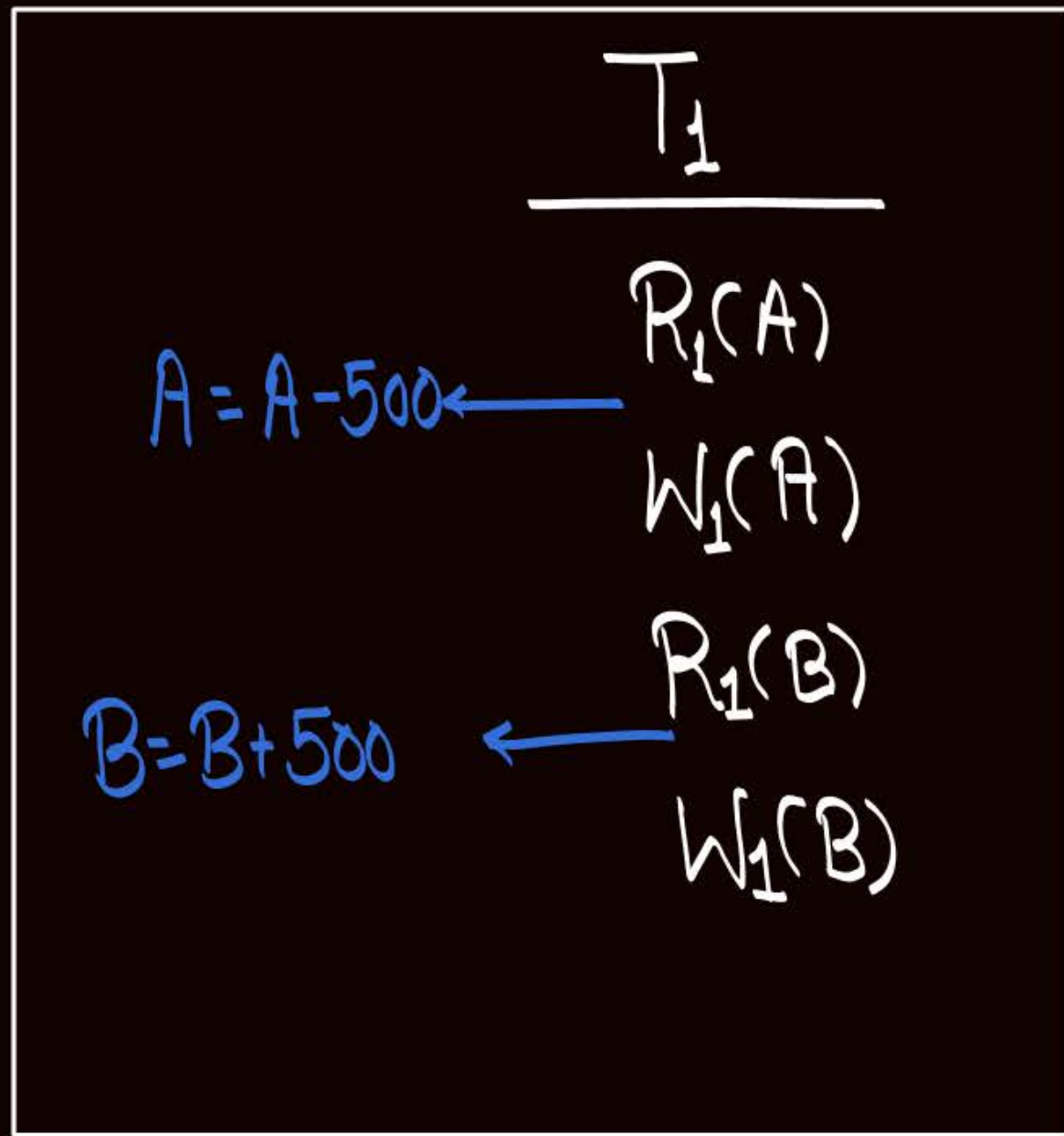


Isolation says that if two or more transactions are executing Concurrently, then they must be unaware of each other.

Consider two transactions,

T₁: Transfer Rs 500/- from account A to account B.

T₂: Read the current balance in account A & account B respectively



$R_i(x)$: It represent read of data item x by transaction T_i

$W_j(y)$: It represent Write of data item y by transaction T_j



Topic : Schedule

Time ordered sequence of operations of two or more transaction is called schedule.

eg:

S	
T ₁	T ₂
R ₁ (A)	
	R ₂ (A)
W ₁ (A)	
	R ₂ (B)
R ₁ (B)	
W ₁ (B)	

time
increasing
↓

$\equiv S = R_1(A); R_2(A); W_1(A); R_2(B); R_1(B); W_1(B);$

→ Consider the following schedule

$R_2(A); W_3(B); R_1(B); W_2(A); R_3(A); W_1(B); \equiv$

T_1	T_2	T_3
	$R_2(A)$	
		$W_3(B)$
$R_1(B)$		
	$W_2(A)$	
		$R_3(A)$
$W_1(B)$		



Topic : Schedule



There are two types of Schedules:

- ① Serial Schedule (i.e., one after another)
- ② Concurrent Schedule (i.e., interleaved execution)



Topic : Serial Schedule

- * If we can start the execution of a new transaction only when the previously started transaction has completed execution, then such schedules are called serial schedules.

eg. Consider two transactions
 $T_1: R_1(A), W_1(A), R_1(B), W_1(B)$
 $T_2: R_2(A), R_2(B)$

If there are two transactions, then only two serial schedules are possible

(i) Execute T_1 , then execute T_2

$T_1 \xrightarrow{|||} T_2$

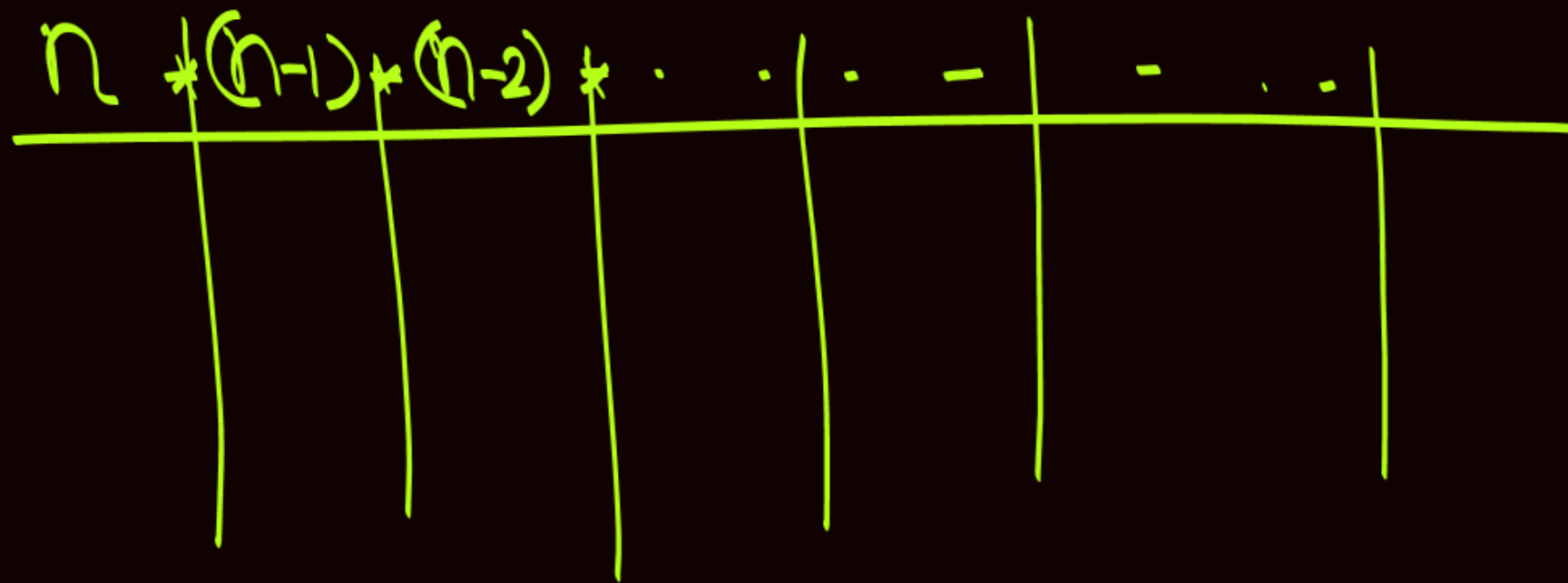
T_1	T_2
$R_1(A)$	
$W_1(A)$	
$R_1(B)$	
$W_1(B)$	
	$R_2(A)$
	$R_2(B)$

(ii) Execute T_2 , then execute T_1

$T_2 \xrightarrow{|||} T_1$

T_1	T_2
	$R_2(A)$
	$R_2(B)$
$R_1(A)$	
$W_1(A)$	
$R_1(B)$	
$W_1(B)$	

Note:- If there are 'n' transactions, then $n!$ serial schedules are possible over those transactions





Topic : Serial Schedule



* Serial schedule will always satisfy isolation Condition. (because the operations of transactions are not executing in interleaved manner)

But throughput of the system may be low with serial schedules.

→ ∴ We think about Concurrent Schedules.



Topic : Concurrent Schedule

If operations of transactions can execute in interleaved manner, then such schedules are called Concurrent Schedules

eg. Consider two transactions
 $T_1: R_1(A), W_1(A), R_1(B), W_1(B)$

$T_2: R_2(A), R_2(B)$

and following schedules over transactions T_1 & T_2

S_1

T_1	T_2
$R_1(A)$	
$W_1(A)$	
	$R_2(A)$
	$R_2(B)$
$R_1(B)$	
$W_1(B)$	

S_2

T_1	T_2
$R_1(A)$	
$W_1(A)$	
	$R_2(A)$
$R_1(B)$	
$W_1(B)$	
	$R_2(B)$

eg. Consider two transactions

$T_1: R_1(A), W_1(A), R_1(B), W_1(B)$ { is to transfer 500 from A to B }

$T_2: R_2(A), R_2(B)$ { Read balance in A & B }

let initially $A = 1000$
 $B = 0$

If there are two transactions, then only two serial schedules are possible

(i) Execute T_1 , then execute T_2

$T_1 \xrightarrow{|||} T_2$

T_1	T_2
$A = 1000 \leftarrow R_1(A)$	
$A = 500 \leftarrow W_1(A)$	
$B = 0 \leftarrow R_1(B)$	
$B = 500 \leftarrow W_1(B)$	
	$R_2(A) \text{ --- } A = 500$
	$R_2(B) \text{ --- } B = 500$

(ii) Execute T_2 , then execute T_1

$T_2 \xrightarrow{|||} T_1$

T_1	T_2
	$R_2(A) \text{ --- } A = 1000$
	$R_2(B) \text{ --- } B = 0$
$A = 1000 \text{ --- } R_1(A)$	
$A = 500 \text{ --- } W_1(A)$	
$B = 0 \text{ --- } R_1(B)$	
$B = 500 \text{ --- } W_1(B)$	

Note :- With concurrent schedule throughput of the system will increase, but isolation condition may not be satisfied

(S₁)

	T ₁	T ₂
A = 1000	R ₁ (A)	
A = 500	W ₁ (A)	
		R ₂ (A) — A = 500
		R ₂ (B) — B = 0
	R ₁ (B)	
	W ₁ (B)	

It is neither same as serial schedule T₁ → T₂ nor same as serial schedule T₂ → T₁

then T₂ will be able to identify that it is executing along with T₁

∴ isolation is not satisfied.

(S₂)

	T ₁	T ₂
A = 1000	R ₁ (A)	
A = 500	W ₁ (A)	
		R ₂ (A) — 500
B = 0	R ₁ (B)	
B = 500	W ₁ (B)	
		R ₂ (B) — 500

All the values are exactly same as serial schedule T₁ → T₂

∴ None of the transaction will be able to identify that they are executing along with other transactions.

Hence, isolation is satisfied.

Note :-

In order to say that a schedule 'S' satisfies the isolation Condition, the behavior of schedule S must be equivalent to at least one of the serial schedule w.r.t. transaction of schedule S.



Topic : Isolation



- ▶ In order to satisfy the isolation Condition, behaviour of δ schedule must be equivalent to at least one of the serial δ schedule.



Topic : Serializable schedule

A schedule is said to be serializable if and only if its behaviour is equivalent to at least one of the serial schedule.

- * Every serial schedule is a serializable schedule, but every serializable schedule need not be a serial schedule.



Topic : Isolation



- In order to satisfy the isolation Condition, behaviour of δ schedule must be equivalent to at least one of the serial δ schedule.

(or)

In order to satisfy the isolation Condition, the δ schedule must be a serializable δ schedule.

	<u>S₁</u>	
	T ₁	T ₂
A=1000	R ₁ (A)	
A=500	W ₁ (A)	
		R ₂ (A) — A=500
		R ₂ (B) — B=0
	R ₁ (B)	
	W ₁ (B)	

It is neither same as serial schedule T₁ → T₂ nor same as serial schedule T₂ → T₁

The behaviour of Schedule S₁ is not equivalent to any of the serial schedules over the transactions T₁ & T₂
 ∴ S₁ is not a serializable schedule
 Hence S₁ does not satisfy isolation

	<u>S₂</u>	
	T ₁	T ₂
A=1000	R ₁ (A)	
A=500	W ₁ (A)	
		R ₂ (A) — 500
B=0	R ₁ (B)	
B=500	W ₁ (B)	
		R ₂ (B) — 500

Behaviour of schedule S₂ is not equivalent to serial schedule T₂ → T₁
 i.e. S₂ ≠ T₂ → T₁
 But schedule S₂ is equivalent to serial schedule T₁ → T₂
 i.e. S₂ ≡ T₁ → T₂
 Schedule S₂ is equivalent to at least one of the serial schedules
 ∴ S₂ is a serializable schedule
 Hence S₂ satisfies isolation Condⁿ.

Note :- Concurrency Control Component of DBMS is responsible for avoiding the execution of non-serializable schedule.

⇓
i.e. Concurrency Control Component ensures serializability

⇓
i.e. Concurrency Control Component ensures isolation.



Topic : Consistency



Consistency states that before and after the execution of schedule the database must remain Consistent

• Criteria for Consistency:-

- ① Schedule must be strict recoverable schedule
- and ② Schedule must be serializable schedule

Note:

- ① For atomicity & durability, Recovery management Comes into the picture
- ② For isolation, Concurrency Control Component Comes into the picture
- ③ For consistency, both recovery management Component and Concurrency Control Component Comes into the picture.



Topic : Equivalent schedule

For two schedules to be Equivalent. following two conditions must satisfy.

① Every corresponding read operation in both the schedule must be identical.

And ② Final Update (Final write opⁿ) for every corresponding data item must be identical in both the schedules

Home Work:-

Consider two transactions:

$T_1: R_1(A), W_1(A), R_1(B), W_1(B)$

$T_2: R_2(A), R_2(B)$

How many total schedules are possible
over the transactions T_1 & T_2 .



2 mins Summary



✓
Topic

Isolation

✓
Topic

Schedule

✓
Topic

Equivalent schedule

✓
Topic

Serializable schedule

✓
Topic

Consistency

THANK - YOU